

CHAPTER 4: DESIGN CONCEPTS AND PRINCIPLES

❖ SOFTWARE DESIGN:

It is the process of creating a detailed plan or blueprint from user requirements to guide software construction and implementation.

Software design is the core technical part of software engineering. It is used no matter which software process model (waterfall, agile, etc.) is followed. After the software requirements are analyzed and clearly defined, software design is the first main technical step. The three main technical activities to build software are design, coding, and testing. Design comes first because it provides a plan or blueprint that guides programmers during coding and helps testers verify the software correctly.

Each part of the analysis model gives the information needed to create the four design models that fully describe the software design. Design is a clear engineering blueprint of what will be built. It is directly linked to the customer's requirements, and it can also be checked for quality using standard rules of good design.

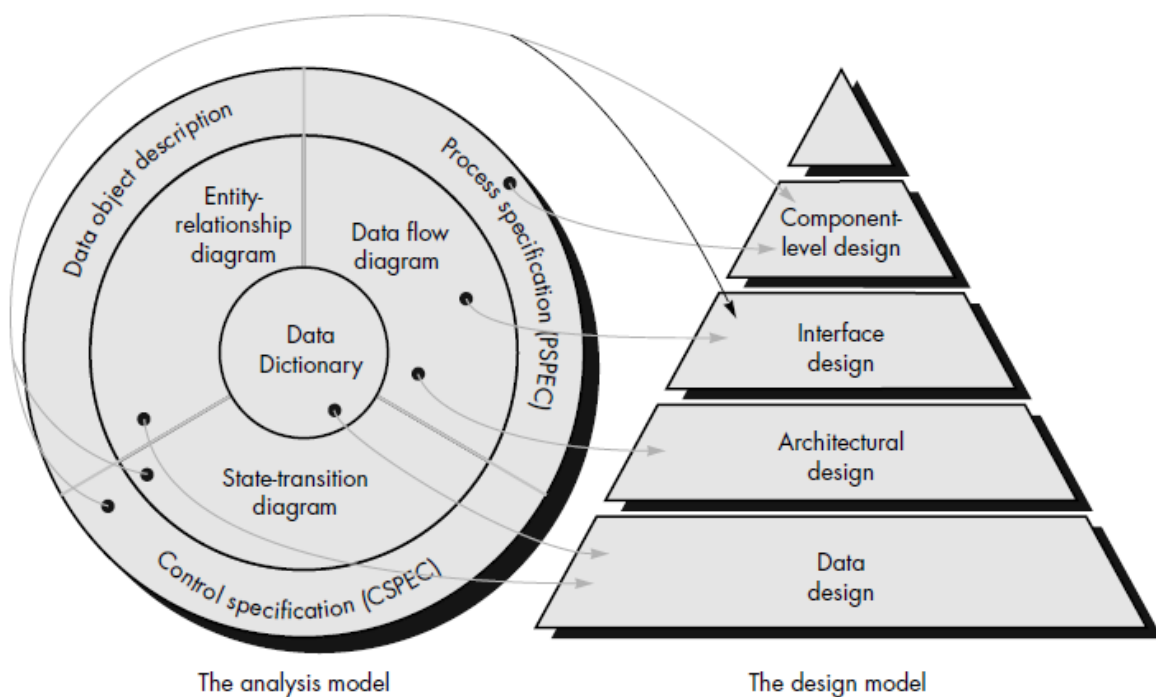


Fig: Translating the analysis model into a software design

In software engineering, **four types of software design**:

- **Data design**
- **Architectural design**

- **Interface design**
- **Component-level design**

The **data design** transforms the information domain model created during analysis into the data structures that will be required to implement the software. The data objects and relationships defined in the entity relationship diagram and the data dictionary provide the basis for the data design activity. It focuses on data structure and database design.

The **architectural design** defines the relationship between major structural elements of the software, the “design patterns” that can be used to achieve the requirements that have been defined for the system, and the constraints that affect the way in which architectural design patterns can be applied. It defines the **overall structure** of the software.

The **interface design** describes how the software communicates within itself, with systems that interoperate with it, and with humans who use it. An interface implies a flow of information (e.g., data and/or control) and a specific type of behavior. Therefore, data and control flow diagrams provide much of the information required for interface design.

The **component-level design** transforms structural elements of the software architecture into a procedural description of software components. Information obtained from the PSPEC (Process Specification), CSPEC (Control Specification), and STD (State Transition Diagram) serve as the basis for component design.

Component-Level Design is the bridge between the high-level architectural design and the actual coding phase. It focuses on defining the internal logic, data structures, and interfaces of the individual "building blocks" (components) that make up a system. **Component-level design turns ideas and diagrams into clear instructions for coding, while ensuring the software is easy to build and maintain.**

★ **Objectives of Software Design:**

- 1. Correctness:** Software design should be correct as per requirement.
- 2. Completeness:** The design should have all components like data structures, modules, and external interfaces, etc.

- 3. Efficiency:** Resources should be used efficiently by the program.
- 4. Flexibility:** Able to modify on changing needs.
- 5. Consistency:** There should not be any inconsistency in the design.
- 6. Maintainability:** The design should be so simple so that it can be easily maintainable by other designers

★ **Why is design so important?**

The importance of software design can be stated with a single word—**quality**.

- Design is the place where quality is fostered in software engineering.
- Design provides us with representations of software that can be assessed for quality.
- Design is the only way that we can accurately translate a customer's requirements into a finished software product or system.
- Software design serves as the foundation for all the software engineering and software support steps that follow.

Without design, we risk building an unstable system—one that will fail when small changes are made; one that may be difficult to test; one whose quality cannot be assessed until late in the software process, when time is short and many dollars have already been spent.

❖ **DESIGN PROCESS, PRINCIPLES AND CONCEPTS**

❖ **THE DESIGN PROCESS:**

The design *process* is a sequence of steps that enable the designer to describe all aspects of the software to be built.

Software design is an iterative process through which requirements are translated into a “blueprint” for constructing the software. The design is represented at a high level of abstraction—a level that can be directly traced to the specific system objective and more detailed data, functional, and behavioral requirements. As design iterations occur, subsequent refinement leads to design representations at much lower levels of abstraction.

Design and Software Quality

Throughout the design process, the quality of the evolving design is assessed with a series of formal technical reviews or design walkthroughs.

Three characteristics that serve as a guide for the evaluation of a good design suggested by McGlaughlin:

- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- The design must be a readable, understandable guide for those who generate code and for those who test and support the software.
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation view.

Following guidelines for good design:

1. A design should exhibit an architectural structure that
 - Has been created using recognizable design patterns,
 - Is composed of components that exhibit good design characteristics.
 - Can be implemented in an evolutionary fashion.
2. A design should be modular; that is, the software should be logically partitioned into elements that perform specific functions and sub functions.
3. A design should contain distinct representations of data, architecture, interfaces, and components (modules).
4. A design should lead to data structures that are appropriate for the objects to be implemented and are drawn from recognizable data patterns.
5. A design should lead to components that exhibit independent functional characteristics.
6. A design should lead to interfaces that reduce the complexity of connections between modules and with the external environment.
7. A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.

The software design process encourages good design through the application of fundamental design principles, systematic methodology, and thorough review.

❖ DESIGN PRINCIPLES:

Software design is both a process and a model. The design *process* is a sequence of steps that enable the designer to describe all aspects of the software to be built.

The design model is the equivalent of an architect's plans for a house. It begins by representing the totality of the thing to be built and slowly refines the thing

to provide guidance for constructing each detail. Similarly, the design model that is created for software provides a variety of different views of the computer software. Creative skill, past experience, a sense of what makes “good” software and an overall commitment to quality are critical success factors for a competent design.

Davis suggests a set of principles for software design, which have been adapted and extended in the following list:

1. The design process should not suffer from “tunnel vision.”

A good designer should consider alternative approaches, judging each based on the requirements of the problem, the resources available to do the job, and the design concepts. A good designer should not think of only one solution. They should think of many possible solutions and

2. The design should be traceable to the analysis model.

Because a single element of the design model often traces to multiple requirements, it is necessary to have a means for tracking how requirements have been satisfied by the design model. Overall, its define that each part of the design must be clearly linked to requirements so we can track how every requirement is fulfilled by the design.

3. The design should not reinvent the wheel.

Systems are constructed using a set of design patterns, many of which have likely been encountered before. These patterns should always be chosen as an alternative to reinvention. Time is short and resources are limited! Design time should be invested in representing truly new ideas and integrating those patterns that already exist. Do not create everything from scratch. Use existing design patterns and solutions whenever possible.

4. The design should “minimize the intellectual distance” between the software and the problem as it exists in the real world.

That is, the structure of the software design should (whenever possible) mimic the structure of the problem domain. The software structure should be similar to the real-world problem.

5. The design should exhibit uniformity and integration.

A design is uniform if it appears that one person developed the entire thing. Rules of style and format should be defined for a design team before design work begins. A design is integrated if care is taken in defining interfaces between design components. The design should look like it was made by one person (same style, rules). All parts should work well together with clear interfaces.

6. The design should be structured to accommodate change.

A good design should be flexible, so future changes can be made easily without breaking the system.

The software should be designed in such a way that it accommodates the change implying that the software should adjust to the change that is required to be done as per the user's need.

7. The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered.

Well-designed software should never “bomb.” It should be designed to accommodate unusual circumstances, and if it must terminate processing, do so in a graceful manner. Software should not suddenly crash. It should handle wrong input or unusual situations smoothly and stop safely if needed.

8. Design is not coding; coding is not design.

Even when detailed procedural designs are created for program components, the level of abstraction of the design model is higher than source code. The only design decisions made at the coding level address the small implementation details that enable the procedural design to be coded.

Design = planning (high level)

Coding = writing actual program instructions (low level)

Coding only handles small implementation details.

9. The design should be assessed for quality as it is being created, not after the fact.

A variety of design concepts and design measures are available to assist the designer in assessing quality. Design quality should be checked while designing, not after everything is done.

10. The design should be reviewed to minimize conceptual (semantic) errors.

There is sometimes a tendency to focus on minutiae when the design is reviewed, missing the forest for the trees. A design team should ensure that major conceptual elements of the design (omissions, ambiguity, and inconsistency) have been addressed before worrying about the syntax of the design model.

Design review should focus on major issues like missing features, unclear concepts, and inconsistencies. Not only focus on minor error like syntax error.

❖ DESIGN CONCEPTS:

Design concepts are principles that help transform software requirements into a structured, efficient, and maintainable design. A set of fundamental software design concepts has evolved over the past four decades. Each provides the software designer with a foundation from which more sophisticated sign methods can be applied. Each helps the software engineer to answer the following questions:

- What criteria can be used to partition software into individual components?
- How function or data is structure detail separated from a conceptual representation of the software?
- What uniform criteria define the technical quality of a software design?

Fundamental design Concepts provide the necessary framework for effective and well-structured software design. These are included as design concepts because they provide basic rules and thinking methods that guide a designer to create good-quality software.

❖ Fundamental design Concepts /Core Design Concepts are:

1. ABSTRACTION:

Abstraction is a design concept that hides implementation details and represents a system at different levels of detail to simplify software design. Abstraction is a fundamental design concept that allows a problem to be viewed at different levels of detail. At a high level of abstraction, the solution is described in broad, problem-oriented terms. As the design progresses, lower levels of abstraction introduce more procedural details. The lowest level of abstraction provides a description that can be directly implemented as source code.

- **Procedural abstraction** refers to a named sequence of instructions that performs a specific function. For example, the operation *open* represents a series of detailed steps required to open a door, without exposing those steps.
- **Data abstraction** is a named collection of data that describes an object. For example, a *door* can be represented by attributes such as type, size, weight, and opening mechanism. Procedural abstractions operate using these data attributes.
- **Control abstraction** represents a control mechanism without revealing its internal implementation. An example is a **semaphore**, which is used for synchronization in operating systems.

2. REFINEMENT:

Refinement is the process of elaborating a high-level design into detailed procedural steps during software design. Refinement causes the designer *to elaborate on the original statement*, providing more and more detail as each successive refinement occurs. Refinement works together with **abstraction**. While abstraction hides low-level details in the early stages of design, refinement progressively reveals those details as the design evolves. By using both abstraction and refinement, a complete and implementable design model is achieved.

Example: "Process payment" is refined into steps like validate card → check balance → deduct amount → generate receipt.)

3. MODULARITY:

"The degree to which software can be understood by examining its components independently of one another."

Software is divided into separately named and addressable components, often called *modules* that are integrated to satisfy problem requirements. Modularity is the single attribute of software that allows a program to be intellectually manageable.

Example: Separate modules for Login, Payment, Report, and User Management in a system.

Five criteria that enable us to evaluate a design method with respect to its ability to define an effective modular system:

Modular decomposability: If a design method provides a systematic mechanism for decomposing the problem into sub problems, it will reduce the complexity of the overall problem, thereby achieving an effective modular solution.

Modular composability: If a design method enables existing (reusable) design components to be assembled into a new system, it will yield a modular solution that does not reinvent the wheel.

Modular understandability: If a module can be understood as a standalone unit (without reference to other modules), it will be easier to build and easier to change.

Modular continuity: If small changes to the system requirements result in changes to individual modules, rather than system wide changes, the impact of change-induced side effects will be minimized.

Modular protection: If an aberrant(error) condition occurs within a module and its effects are constrained within that module, the impact of error-induced side effects will be minimized.

4. Software Architecture:

Software architecture refers to “the overall structure of the software and the ways in which that structure provides conceptual integrity for a system” Architecture is the hierarchical structure of program components (modules), the manner in which these components interact and the structure of data that are used by the components.

A set of properties that should be specified as part of an architectural design:

Structural properties: This aspect of the architectural design representation defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another. For example, objects are packaged to encapsulate both data and the processing that manipulates the data and interact via the invocation of methods.

Extra-functional properties: The architectural design description should address how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.

Families of related systems: The architectural design should draw upon repeatable patterns that are commonly encountered in the design of families of similar systems. In essence, the design should have the ability to reuse architectural building blocks.

Given the specification of these properties, the architectural design can be represented using one or more of a number of different models.

Structural models represent architecture as an organized collection of program components.

Framework models increase the level of design abstraction by attempting to identify repeatable architectural design frameworks (patterns) that are encountered in similar types of applications.

Dynamic models address the behavioral aspects of the program architecture, indicating how the structure or system configuration may change as a function of external events.

Process models focus on the design of the business or technical process that the system must accommodate.

Functional models can be used to represent the functional hierarchy of a system.

A number of different *architectural description languages* (ADLs) have been developed to represent these models. Although many different ADLs have been proposed, the majority provide mechanisms for describing system components and the manner in which they are connected to one another. Examples of ADLs include **Wright, Acme, Rapide etc.**

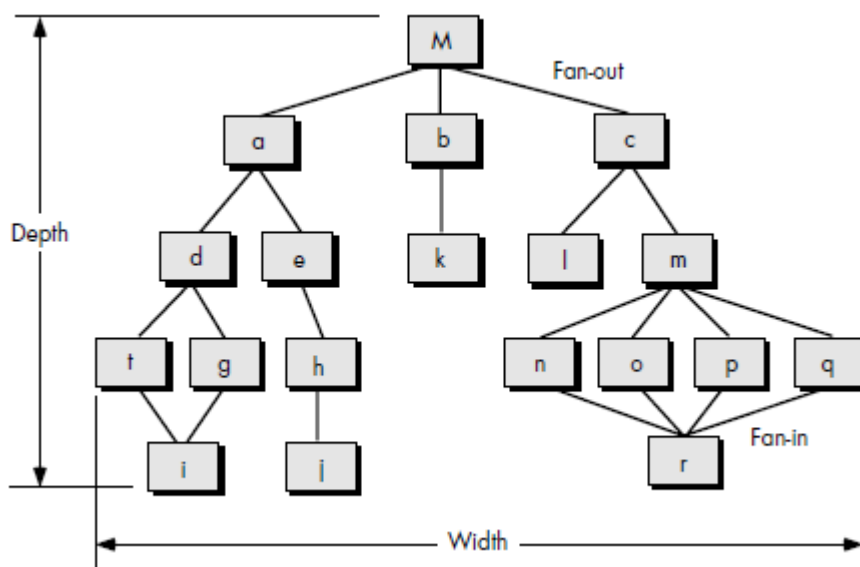
5. Control Hierarchy:

It Shows how control flows between modules in a hierarchical structure.

Control hierarchy, also called *program structure*, represents the organization of program components (modules) and implies a hierarchy of control. It does not represent procedural aspects of software such as sequence of processes. **Control hierarchy** is important because it organizes modules, reduces complexity, supports maintenance, testing, and reuse, and improves understanding of system structure.

Different notations are used to represent control hierarchy for those architectural styles that are amenable to this representation.

The most common is the tree-like diagram that represents hierarchical control for call and return architectures.



Depth and *width* provide an indication of the number of levels of control and overall *span of control*, respectively.

Fan-out is a measure of the number of modules that are directly controlled by another module.

Fan-in indicates how many modules directly control a given module.

The control hierarchy also represents two different characteristics of the software architecture: **visibility and connectivity**.

Visibility: All components or data that a module can access, even indirectly.

Example: A module may see many inherited objects but use only a few.

Connectivity: Components or data that a module directly uses or interacts with.

Example: A module directly calling another module is connected to it.

6. Structural Partitioning:

Structural partitioning is the process of dividing a software system into smaller, manageable modules or components to simplify design, implementation, and maintenance. If the architectural style of a system is hierarchical, the program structure can be partitioned both horizontally and vertically. **Organizes the system by functions (horizontal) and control levels (vertical).**

- **Horizontally (by function):** Separate the system into main functions like input, processing, and output.
- **Vertically (by control):** Arrange modules so top modules handle decisions (control) and lower modules do the actual work (processing).
This makes the system easier to understand, change, and maintain, and keeps modules organized and independent.

7. Functional Independence:

Functional independence is a key to good design, and design is the key to software quality. The concept of functional independence is a direct outgrowth of separation of concerns, modularity, and the concepts of abstraction and information hiding. Functional independence is achieved by developing modules with “single minded” function and an “aversion” to excessive interaction with other modules. Independent modules, is easier to develop because function can be compartmentalized and interfaces are simplified. Independent modules are easier to maintain (and test) because secondary effects caused by design or code modification are limited, error propagation is reduced, and reusable modules are possible.

Example: A Login module only handles authentication and does not manage payments or reports.

Independence is assessed using **two qualitative criteria: cohesion and coupling**.

Coupling and cohesion are fundamental concepts in software design that help in creating modular, maintainable, and scalable software systems.

- **Cohesion** refers to the degree to which the elements within a module belong together. A highly cohesive module performs a single, well-defined task.

- **Coupling** refers to the degree of interdependence between modules. A loosely coupled system minimizes interdependence, enhancing maintainability and scalability.

8. Refactoring:

It Improves design or code structure without changing functionality.

An important design activity suggested for many agile methods, refactoring is a reorganization technique that simplifies the design (or code) of a component without changing its function or behavior. When software is refactored, the existing design is examined for redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failure that can be corrected to yield a better design.

Example: Removing duplicate code and creating a common function without changing program output.

9. Information Hiding:

Modules should be designed in such a way that the internal data and procedures of a module are not accessible to other modules unless it is necessary. This concept is known as **information hiding**. It helps in achieving effective modularity by allowing modules to work independently and communicate only through essential information required to perform software functions.

Abstraction is used to describe the important features of a module while hiding unnecessary implementation details. It defines what a module does without explaining how it does it. On the other hand, **information hiding** controls access to the internal procedures and data structures of a module and prevents other modules from directly using them.

Information hiding is especially beneficial during software testing and maintenance. Since most internal details are hidden, changes made to one module are less likely to affect other parts of the software. This reduces the

chances of introducing errors and makes the software easier to modify and maintain.

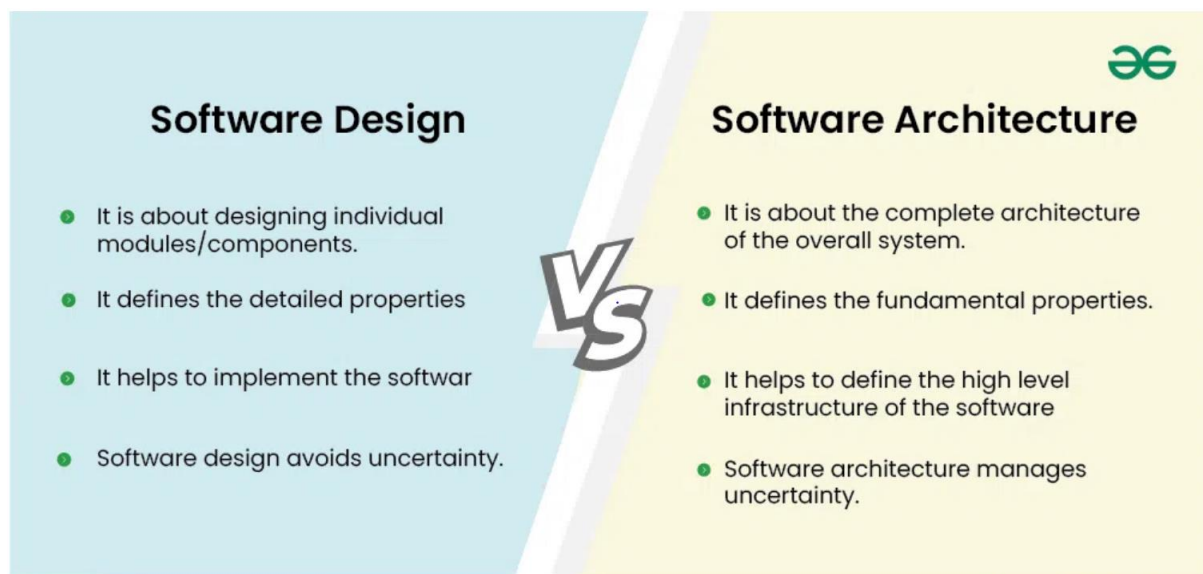
❖ SOFTWARE ARCHITECTURE:

Design has been described as a multistep process in which representations of data and program structure, interface characteristics, and procedural detail are synthesized from information requirements.

Software design and software architecture, though closely related, serve different purposes in system development.

Software architecture provides a high-level view of a software system, focusing on the organization of major modules, their interactions, and the overall system structure. It defines the structural framework that ensures conceptual integrity, modularity, and maintainability of the system.

Software design focuses on the detailed implementation of individual components and their interactions, guiding how the code is structured and written. This includes component-level design, which defines the internal logic, procedures, and algorithms of individual modules, as well as data design, which determines how data is organized, structured, and accessed within the system. Together, architecture and design provide a complete blueprint that guides the implementation and ensures that the system meets its functional and non-functional requirements.



Software architecture considers two levels of the design Pyramid:

1. Architectural Level Design
2. Component Level Design

1. Architectural level Design (High level):

Architectural design represents the high-level structure of program components and data required to build a computer-based system. It is the process of defining the overall structure of a system. It establishes one or more representations of the system architecture, which are then used to guide data design and component-level design.

The IEEE definition:

“The process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.”

❖ Purpose of Architectural-Level Design:

- **Define the system structure** – Identify major components, modules, and their interactions.
- **Provide a high-level blueprint** – Serve as a guide for detailed design and implementation.
- **Select architectural style** – Choose an appropriate pattern (layered, client-server, object-oriented, etc.).
- **Manage system complexity** – Break down a large system into smaller, manageable parts.
- **Facilitate communication** – Help developers, designers, and stakeholders understand the system.
- **Ensure system quality** – Support attributes like performance, reliability, scalability, and maintainability.
- **Reduce development risk** – Identify potential design issues and bottlenecks early in the project.

2. Component Level Design (Low level/ detailed module design):

Component-Level Design is the detailed design of individual software components, specifying their internal logic, data structures, and methods, based on the architectural and data design.

The component-level design for software fully describes the internal detail of each software component. To accomplish this, the component-level

design defines data structures for all local data objects and algorithmic detail for all processing that occurs within a component and an interface that allows access to all component operations (behaviors).

Component-level design uses information from **PSPEC**, **CSPEC**, and **STD** to decide how each software component will be implemented.

- **PSPEC (Process Specification)** provides details about what processing must be done. It helps design the internal logic and algorithms of a component. Example: Validate username and password.
- **CSPEC (Control Specification)** describes how control flows in the system and when processes are executed. It helps design control components and decision-making logic.

Example: If login is valid → open dashboard, else → show error.

- **STD (State Transition Diagram)** shows different states of the system and transitions between them. It helps design state-dependent components and their behavior.

Example: States like Idle → Verifying → Logged In / Failed. (system behavior through different states).

❖ **Purpose of component level design:**

1. **Translate design into code**

- Converts architectural design into detailed module logic.
- Helps programmers know exactly what to code.

2. **Define internal details of components**

- Specifies algorithms, data structures, and control logic.
- Clearly defines inputs, outputs, and processing steps.

3. **Improve understandability**

- Makes complex systems easier to understand by breaking them into small, manageable components.

4. **Support maintainability and reusability**

- Well-designed components are easier to modify, test, and reuse.

5. **Reduce errors during coding**

- Clear design reduces ambiguity and coding mistakes.

6. **Ensure traceability**

- Ensures every component is traceable back to requirements and analysis artifacts (PSPEC, CSPEC, STD).

7. Enable independent development and testing

- Each component can be developed and tested separately.

★ Architectural level design vs component level design:

Aspect	Architectural-Level Design	Component-Level Design
1. Scope	Defines the overall structure of the software system.	Focuses on the detailed design of individual components.
2. Abstraction Level	Operates at a high level of abstraction.	Operates at a low level of abstraction.
3. Focus	Emphasizes system-wide concerns like communication and data flow.	Emphasizes internal logic and functionality of components.
4. Goals	Ensures system meets non-functional requirements like scalability, performance, and reliability.	Implements the specific functional requirements of the system.
5. Tools/Diagrams Used	Uses high-level diagrams like system context, component, and deployment diagrams.	Uses detailed diagrams like class, sequence, and state diagrams.
6. Reusability	Focuses on defining reusable subsystems or layers.	Focuses on designing reusable methods or classes.
7. Communication	Defines how major subsystems or layers communicate.	Defines how functions or methods interact within a component.
8. Decision Impact	Impacts the overall system's scalability and maintainability.	Impacts the functionality and correctness of specific parts.

❖ DATA DESIGN:

Data design is the process of organizing and structuring data so software can use it efficiently. Data design defines how data is structured, stored, and shared in the system.

Data design creates a model of data and/or information that is represented at a high level of abstraction (the customer/user's view of data).

It transforms high-level user/customer data requirements into practical data structures, databases, or data warehouses.

Data structure is a very important part of software design. Data design plays a key role at three levels.

- At the **program component level**, it focuses on designing proper data structures and algorithms, which helps in building efficient and high-quality software.
- At the **application level**, data design converts the data model into a database, which is essential for meeting the system's business requirements.
- At the **business level**, data from different databases is collected into a data warehouse, which supports data mining (knowledge discovery) to find useful patterns and trends that help business decision-making.

Data mining: Data mining is the process of analyzing large sets of data to discover useful patterns, trends, or knowledge that is not immediately obvious.

Data warehouse: A data warehouse is a large storage system that collects historical and summarized data from multiple sources to support analysis, reporting, and decision-making.

At the architectural level, it helps decide what data different modules will use and how data will flow between them. This ensures that components can communicate properly using well-defined data formats.

At the component level, data design provides detailed data structures, such as variables, records, or objects, that are used inside each module to implement its logic.

In simple words, data design acts as a bridge: it supports architectural design by organizing data for interaction between components and supports component-level design by supplying the data needed for internal processing.

Component-Level Data Design Principles

1. Treat data systematically like functions.
2. Identify all data structures and operations on them.
3. Use a data dictionary to define data.

4. Make low-level decisions later in the design.
5. Only modules that need the data should access it.
6. Maintain a library of reusable data structures.
7. Support abstract data types in design and programming.

❖ **ARCHITECTURAL STYLES:**

An architectural style describes a category of software systems that share a common organization pattern. Each architectural style defines how a system is structured and how its parts interact. Each style describes a system category that encompasses:

1. A set of components (e.g., a database, computational modules) that perform a function required by a system;
2. A set of connectors that enable “communication, co ordinations and cooperation” among components;
3. Constraints that define how components can be integrated to form the system; and
4. Semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts. Semantic models mean models that show the meaning and behavior of a system, not just its structure or flow.

Commonly used architectural pattern:

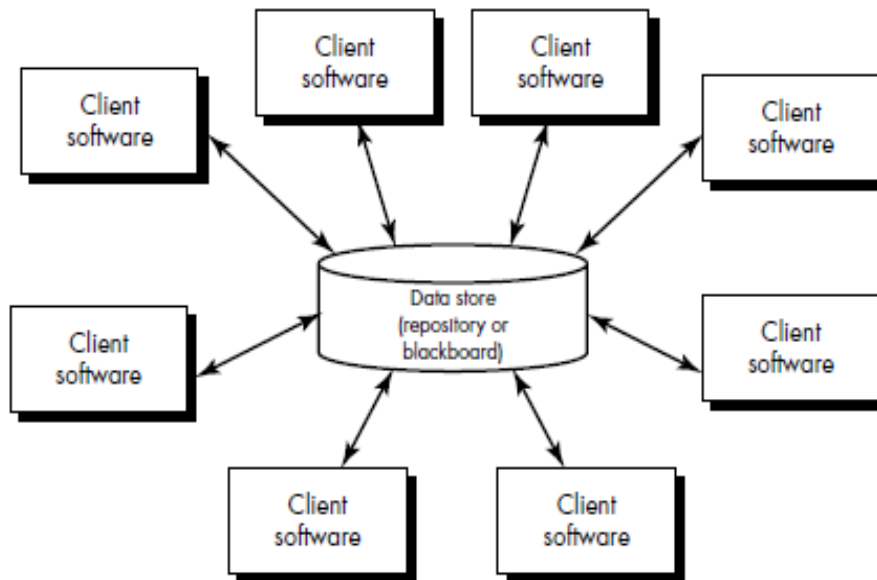
1. Data Centered Architecture:

A data store (e.g., a file or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store. Here client software accesses the data independent of any changes to the data or the actions of other client software.

In some cases the data repository is *passive*. That is, client software accesses the data independent of any changes to the data or the actions of other client software.

Data can also be passed among clients using blackboard mechanism that is sends notifications to client software when data of interest to the client change.

Data-centered architectures promote *integrability*. That is, existing components can be changed and new client components can be added to the architecture without concern about other clients.



2. Data-flow architectures:

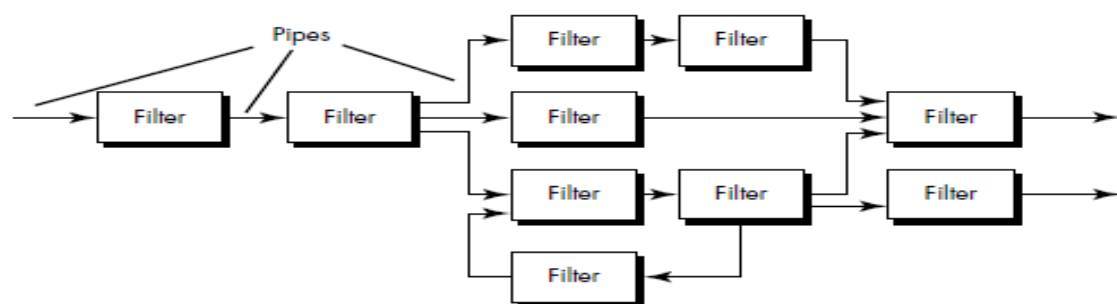
This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data.

A **pipe and filter pattern** has a set of components, called *filters*, connected by pipes that transmit data from one component to the next and each filter works independently.

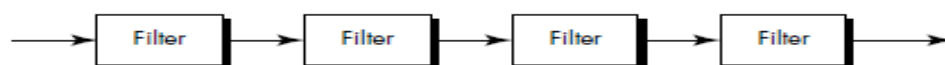
However, the filter does not require knowledge of the working of its neighboring filters.

If the data flow degenerates into a single line of transforms, it is termed **batch sequential**.

This pattern accepts a batch of data and then applies a series of sequential components (filters) to transform it.



(a) Pipes and filters



(b) Batch sequential

Filter: Transforming data independently

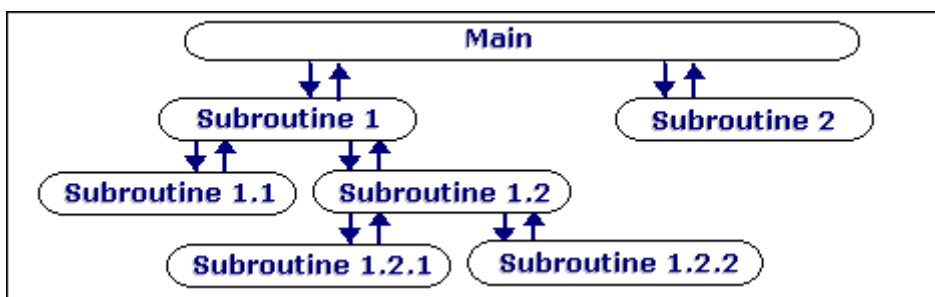
pipe: transmitting data

3. Call and return architectures:

This architectural style enables a software designer (system architect) to achieve a program structure that is *relatively easy to modify and scale*.

Sub styles of this type:-

- **Main program/subprogram architectures:** This classic program structure decomposes function into a control hierarchy where a “main” program invokes a number of program components, which in turn may invoke still other components.
- **Remote procedure call architectures:** The components of a main program/subprogram architecture are distributed across multiple computers on a network.



4. Object-oriented architectures:

The components of a system encapsulate data and the operations that must be applied to manipulate the data. Communication and coordination between components is accomplished via message passing.

5. Layered architectures:

In this architectural style, numbers of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set.

At the outer layer, components service user interface operations.

At the inner layer, components perform operating system interfacing.

Intermediate layers provide utility services and application software functions.

Utility layers, provides **common services** used by the application

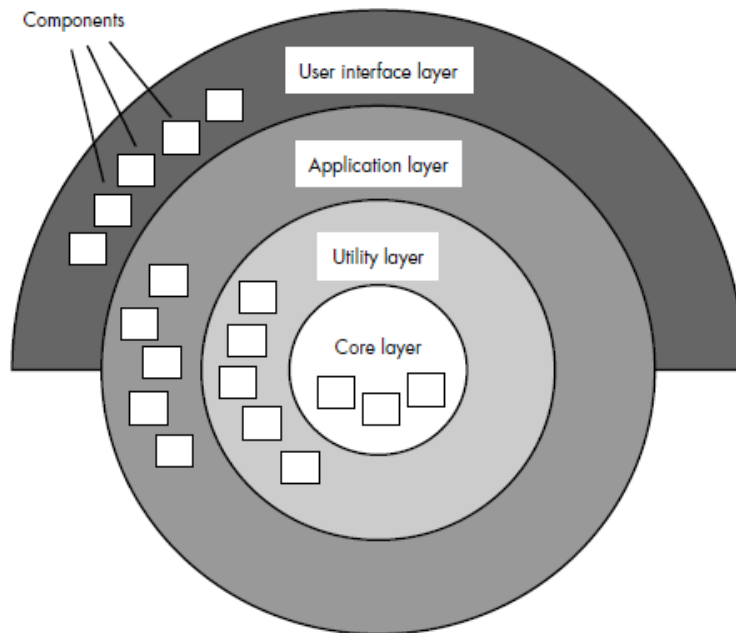


Fig: Layered Architecture

❖ **MAPPING REQUIREMENTS INTO A SOFTWARE ARCHITECTURE**

- The goal is to convert software requirements into a structured design showing modules and their interactions.
- Software requirements can be mapped into various representations of the design model.
- The mapping technique, sometimes called structured design, has its origins in earlier design concepts that stressed modularity, top-down design, and structured programming.
- Structured design is often characterized as a data flow-oriented design method because it provides a convenient transition from a data flow diagram to software architecture.

Six-Step Process to Map DFD to Program Structure

- 1. Establish the type of information flow**
 - Decide whether the data moves in a transform flow (linear processing) or transaction flow (a single input triggers multiple paths).
- 2. Indicate flow boundaries**
 - Identify where data enters and exits the system.
 - Define the limits of processing modules.
- 3. Map the DFD into program structure**
 - Convert the DFD processes (bubbles) into modules in a hierarchical structure.

4. Define the control hierarchy

- Decide which modules are top-level (decision/control) and which are low-level (processing/input/output).

5. Refine the structure using design measures and heuristics

- Apply principles like cohesion, coupling, and module independence to improve design quality.

6. Refine and elaborate the architectural description

- Finalize the architecture, making sure it is clear, maintainable, and implementable.

Types of Data Flow:

1. Transform Flow

Recalling the fundamental system model (level 0 data flow diagram), information must enter and exit software in an "external world" form.

For example, data typed on a keyboard, tones on a telephone line, and video images in a multimedia application are all forms of external world information.

Such externalized data must be converted into an internal form for processing. Information enters the system along paths that transform external data into an internal form. These paths are identified as incoming flow. At the kernel of the software, a transition occurs. Incoming data are passed through a transform center and begin to move along paths that now lead "out" of the software. Data moving along these paths are called outgoing flow. The overall flow of data occurs in a sequential manner and follows one, or only a few, "straight line" paths.

When a segment of a data flow diagram exhibits these characteristics, transform flow is present.

2. Transaction Flow

The fundamental system model implies transform flow; therefore, it is possible to characterize all data flow in this category. However, information flow is often characterized by a single data item, called a transaction that triggers other data flow along one of many paths. When a DFD takes the form, transaction flow is present. Transaction flow is characterized by data moving along an incoming path that converts external world information into a transaction. The transaction is evaluated and, based on its value; flow along one of many action paths is initiated. The hub of information flow from which many action paths emanate is called a transaction center. It should be noted that, within a DFD for a large system, both transform and transaction

flow may be present. For example, in a transaction-oriented flow, information flow along an action path may have transform flow characteristics.

Types of Mapping:

1. TRANSFORM MAPPING:

Transform mapping is a design technique that converts a DFD with transform flow (linear data processing) into a modular and hierarchical program structure. It is useful for designing systems where data enters, is processed, and then exits in a mostly sequential path.

Design Steps:

Step 1: Review the fundamental system model.

The fundamental system model encompasses the level 0 DFD and supporting information. In actuality, the design step begins with an evaluation of both the System Specification and the Software Requirements Specification. So, in this step Analyze the level 0 DFD along with the System Specification and Software Requirements Specification to understand the overall system.

Step 2: Review and refine data flow diagrams for the software. Information obtained from analysis models contained in the Software Requirements Specification is refined to produce greater detail.

Step 3: Determine whether the DFD has transform or transaction flow characteristics.

In this step, the designer selects global (software wide) flow characteristics based on the prevailing nature of the DFD. In addition, local regions of transform or transaction flow are isolated. These subflows can be used to refine program architecture derived from a global characteristic described.

Step 4: Isolate the transform center by specifying incoming and outgoing flow boundaries.

The transforms (bubbles) that constitute the transform center lie within the two shaded boundaries that run from top to bottom in the figure. An argument can be made to readjust a boundary (e.g, an incoming flow boundary separating read sensors and acquire response info could be proposed). The emphasis in this design step should be on selecting reasonable boundaries, rather than lengthy iteration on placement of divisions.

Step 5: Perform "first-level factoring."

Program structure represents a top-down distribution of control. Factoring results in a program structure in which top-level modules perform decision making and low-level modules perform most input, computation, and output work. Middle-level modules perform some control and do moderate amounts of work. When transform flow is encountered, a DFD is mapped to a specific structure (a call and return architecture) that provides control for incoming, transform, and outgoing information processing.

Step 6: Perform "second-level factoring." Second-level factoring is accomplished by mapping individual transforms (bubbles) of a DFD into appropriate modules within the architecture.

Beginning at the transform center boundary and moving outward along incoming and then outgoing paths, transforms are mapped into subordinate levels of the software structure. Two or even three bubbles can be combined and represented as one module (recalling potential problems with cohesion) or a single bubble may be expanded to two or more modules. Practical considerations and measures of design quality dictate the outcome of second level factoring. Review and refinement may lead to changes in this structure, but it can serve as a "first-iteration" design.

Step 7: Refine the first-iteration architecture using design heuristics for improved software quality.

First-iteration architecture can always be refined by applying concepts of module independence. Modules are exploded or imploded to produce sensible factoring, good cohesion, minimal coupling, and most important, a structure that can be implemented without difficulty, tested without confusion, and maintained without grief.

2. TRANSACTION MAPPING:

In many software applications, a single data item triggers one or a number of information flows that affect a function implied by the triggering data item. In this section we consider design steps used to treat transaction flow.

Step 1: Review the fundamental system model.

Step 2: Review and refine data flow diagrams for the software.

Step 3: Determine whether the DFD has transform or transaction flow characteristics.

(Steps 1, 2, and 3 are identical to corresponding steps in transform mapping.)

Step 4: Identify the transaction center and the flow characteristics along each of the action paths.

The location of the transaction center can be immediately discerned from the DFD. The transaction center lies at the origin of a number of actions paths that flow radially from it.

Step 5: Map the DFD in a program structure amenable to transaction processing.

Transaction flow is mapped into an architecture that contains an incoming branch and a dispatch branch. The structure of the incoming branch is developed in much the same way as transform mapping. Starting at the transaction center, bubbles along the incoming path are mapped into modules. The structure of the dispatch branch contains a dispatcher module that controls all subordinate action modules. Each action flow path of the DFD is mapped to a structure that corresponds to its specific flow characteristics.

Step 6: Factor and refine the transaction structure and the structure of each action path.

Each action path of the data flow diagram has its own information flow characteristics. We have already noted that transform or transaction flow may be encountered. The action path-related "substructure" is developed using the design steps discussed in this and the preceding section.

Step 7: Refine the first-iteration architecture using design heuristics for improved software quality.

This step for transaction mapping is identical to the corresponding step for transform mapping. In both design approaches, criteria such as module independence, practicality (efficacy of implementation and test), and maintainability must be carefully considered as structural modifications are proposed.

You refine the structure by checking:

Cohesion: Does each module do only one thing?

Coupling: Are the modules too dependent on each other?

Fan-out: Does one module control too many subordinates?

❖ **STRUCTURED PROGRAMMING:**

Structured programming is a modular approach that improves clarity, quality, and maintainability by organizing code using well-defined control structures instead of using like goto statement.

On the contrary, in the Assembly languages like Microprocessor 8085, etc, the statements do not get executed in a structured manner. It allows jump statements like GOTO. So, the program flow might be random.

The structured program mainly consists of three types of elements:

1. Sequence Statements:

Instructions are executed one after another in order. This is the default behavior of most programming languages.

2. Selection Statements:

- Executes different instructions based on a condition.
- Types: if, if-else, switch.

3. Iteration Statements:

- Repeats a set of instructions until a condition is met.
- Types: for, while, do-while.

The structured program consists of well-structured and separated modules. But the entry and exit in a Structured program is a single-time event. It means that the program uses single-entry and single-exit elements. Therefore, a structured program is well maintained, neat and clean program. This is the reason why the Structured Programming Approach is well accepted in the programming world.

Advantages of Structured Programming

- Easy to understand due to clear and logical flow.
- Easy to maintain because of single-entry and single-exit modules.
- Reduces program complexity by dividing code into modules/functions.
- Encourages code reusability.
- Easier to test and debug.
- Improves readability; code is neat and organized.
- Supports top-down design for systematic planning.

Disadvantages of Structured Programming

- Requires more effort and time during initial design.
- Can be difficult to manage in very large or complex systems.
- May require slightly more memory due to multiple modules/functions.
- Not always suitable for real-time or event-driven systems.

❖ COMPARISON OF DESIGN NOTATION:

Design notation in software engineering refers to the symbolic and graphical representation schemes used to plan, document, and communicate a software system's structure, behavior, and data flow before actual coding begins. They help designers understand, analyze, and communicate the structure and behavior of software effectively.

Common Design Notations:

1. **Unified Modeling Language (UML)** is a standardized modeling language widely used for object-oriented software design. It provides a set of diagrams such as Class, Sequence, Use Case, Activity, and State Machine diagrams to represent the structural, behavioral, and interaction aspects of a system.
2. **Data Flow Diagrams (DFDs)**, also known as bubble charts, focus on the movement of data within a system. They illustrate how data flows from inputs through processing units to outputs. DFDs are primarily used in structured analysis to understand system functionality and data transformation.
3. **Structure Charts (SC)** represent the hierarchical organization of software modules. They show the static structure of a program, including module decomposition, control relationships, and data flow between modules, and are commonly used in structured design.
4. **Pseudocode** is a textual representation of program logic written in a simple, language-independent manner. It bridges the gap between high-level design and actual code by describing algorithms without strict programming syntax.
5. **Flowcharts** are graphical representations of algorithms that depict the flow of control using symbols for processes, decisions, loops, and input/output operations. They are useful for detailed logic design and easy understanding of program flow.

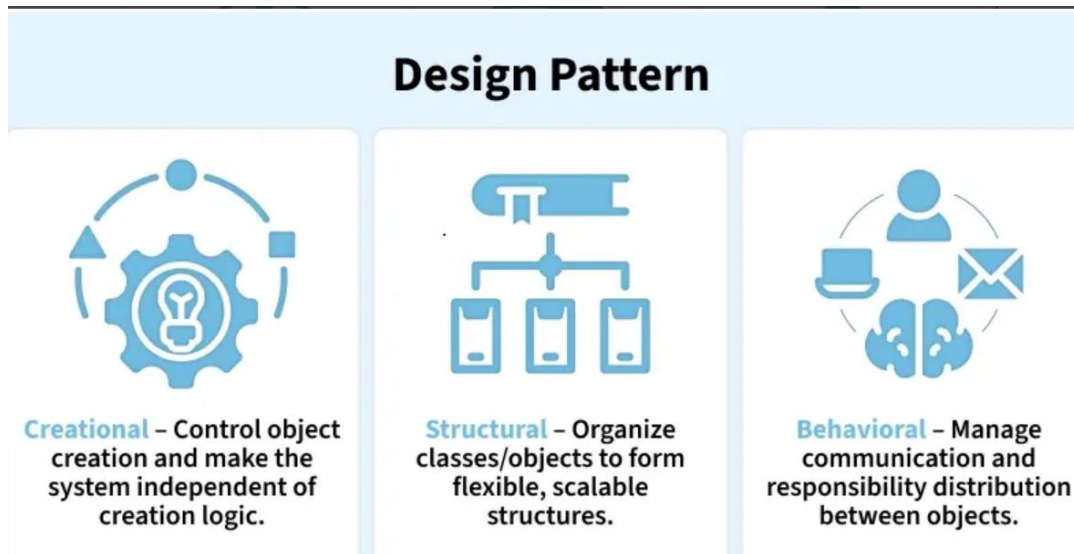
6. HIPO (Hierarchy plus Input-Process-Output) diagrams present a system's functions in a hierarchical structure. They describe the inputs, processing activities, and outputs at each level of the system and are mainly used in structured programming and design.

❖ **INTRODUCTION TO DESIGN PATTERN:**

- In software engineering, a design pattern is a reusable, general solution to a commonly occurring problem in software design.
- It's not a finished piece of code, but rather a template or blueprint for solving a problem in a way that balances flexibility, maintainability, and efficiency.
- Design patterns capture best practices and lessons learned from experienced developers.
- A design pattern describes a problem which occurs over and over again in our environment.
- Then it describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same twice.
- The idea was picked up by four authors: Erich Gamma, John Vlissides, Ralph Johnson, and Richard Helm.
- The book featured 23 patterns solving various problems of object-oriented design and became a best-seller very quickly.

Types of Design Patterns:

Three types of design patterns



1. Creational Design Patterns:

Creational Design Patterns focus on the process of object creation or problems related to object creation. They help in making a system independent of how its objects are created, composed, and represented. Creational patterns give a lot of flexibility in what gets created, who creates it, and how it gets created.

Examples:

- **Singleton:** Ensures only one instance of a class exists.
- **Factory Method:** Creates objects without specifying the exact class.
- **Abstract Factory:** Creates families of related objects.
- **Builder:** Builds complex objects step by step.
- **Prototype:** Creates new objects by copying an existing one.

2. Structural Design Patterns:

Structural design patterns focus on how classes and objects are arranged and connected to form larger, efficient, and easy-to-maintain software

systems. They help simplify complex relationships, reuse existing code, and build scalable architectures.

Examples:

- **Adapter:** Allows incompatible interfaces to work together.
- **Decorator:** Adds responsibilities to objects dynamically.
- **Facade:** Provides a simplified interface to a complex subsystem.
- **Composite:** Treats individual objects and compositions uniformly.
- **Proxy:** Provides a placeholder or surrogate for another object.

3. Behavioral Design Patterns:

Behavioral design patterns are a category of design patterns that focus on the interactions and communication between objects. They help define how objects collaborate and distribute responsibility among them, making it easier to manage complex control flow and communication in a system.

Examples:

- **Observer:** Allows objects to be notified of changes in another object.
- **Strategy:** Defines a family of algorithms and makes them interchangeable.
- **Command:** Encapsulates a request as an object.
- **Iterator:** Provides a way to access elements sequentially.
- **State:** Allows an object to change behavior when its state changes.

Benefits of Design Patterns:

1. Enhances Maintainability

- Patterns organize code in a consistent way, making it easier for developers to understand, update, or extend the software later.

2. Promotes Reusability

- They provide templates or solutions for common design problems that can be reused in different parts of the application or even in other projects.

3. Simplifies Complex Problems

- Complex systems are broken into smaller components, so each problem can be solved independently, making development simpler and more manageable.

4. Improves Scalability

- Structural patterns help build flexible and extensible systems, allowing new features or modules to be added easily without breaking existing code.

5. Improves Testability

- Code designed with patterns is usually modular, so unit tests can be written for individual components, making the software more reliable.

6. Supports Cross-Platform Development

- Design patterns are language and platform independent, so the same solution can be applied in different programming environments.

7. Enhances Collaboration

- Provides a common vocabulary for developers, so teams can communicate clearly about design solutions and work together effectively.

Application of Design Patterns:



Assignments

1. Differentiate between the architectural level design and component level design.
2. Explain the process of transaction mapping into software architecture.
3. Explain the process of transform mapping into software architecture.
4. What are different considerations for effective software design? Explain any three design concepts.
5. What is architecture design? What are the different types of architectural styles?
6. Write down the design principles. Explain how coupling and cohesion helps in effective modular design.
7. Discuss the importance of software architecture? Differentiate between transform mapping and transaction mapping.
8. What is data design? explain software design process.
9. What is software architecture? Why it is important? Describe data centered and data flow architecture models.
10. Define software design. explain different element of design model.

11. Define design notation and design pattern. Explain the types of design pattern.